



날씨 조회 - 프리코스

프리코스 진행 방식

진행 방식

- 미션은 과제 진행 요구 사항, 기능 요구 사항, 프로그래밍 요구 사항 세 가지로 구성되어 있다.
- 세 개의 요구 사항을 만족하기 위해 노력한다. 특히 기능을 구현하기 전에 기능 목록을 만들고, 기능 단위로 커밋하는 방식으로 진행한다.
- 기능 요구 사항에 기재되지 않은 내용은 스스로 판단하여 구현한다.
- 프로그래밍 요구 사항은 단순한 규칙이 아니라 문제를 분해하고 명확하게 설계하는 훈련을 위한 장치임을 이해하고 지킨다.
- AI 도구를 사용할 수 있으나, 문제를 이해하지 않고 단순히 복사해서 붙여 넣는 것은 허용하지 않는다.

미션 제출 방법

- 미션 구현을 완료한 후 GitHub을 통해 제출해야 한다.
 - GitHub을 활용한 제출 방법은 프리코스 진행 가이드 문서를 참고해 제출한다.
 - 풀 리퀘스트 제목은 '[\$이름] 프리코스 미션 제출합니다.'로 작성한다.
- GitHub에 미션을 제출한 후 [구글 폼](#)에 PR 링크를 포함하여 최종 제출한다.

과제 진행 소감

과제를 수행하면서 느낀 점, 배운 점, 많은 시간을 투자한 부분 등 자유롭게 작성한다.

예시

이번 미션은 생각만큼 쉽지 않았습니다.
처음에는 AI를 사용해 초안을 만들었지만 그대로 쓰지 않고 코드를 분석하며 이해하려고 노력하였습니다.
그 과정에서 함수 분리와 테스트 작성의 필요성을 더 잘 알게 되었습니다.
기능을 분리해 기능 단위로 커밋하는 것이 쉽지 않았지만, 연습을 통해 점차 익숙해졌습니다.
결국 더 명확하게 구현할 수 있었고, 미션을 수행하면서 성장한 걸 느꼈습니다. :)

과제 제출 전 체크리스트

- 터미널에서 `java -version` 을 실행하여 Java 버전이 21인지 확인한다. Eclipse 또는 IntelliJ IDEA와 같은 IDE에서 Java 21로 실행되는지 확인한다.
- 터미널에서 Mac 또는 Linux 사용자의 경우 `./gradlew clean test` 명령을 실행하고, Windows 사용자의 경우 `gradlew.bat clean test` 또는 `./gradlew.bat clean test` 명령을 실행할 때 모든 테스트가 아래와 같이 통과하는지 확인한다.

BUILD SUCCESSFUL in 0s

날씨 조회

과제 진행 요구 사항

- 미션은 [날씨 조회](#) 저장소를 포크하고 클론하는 것으로 시작한다.
- 기능을 구현하기 전 `README.md` 에 구현할 기능 목록을 작성하고, 구현 전략을 함께 정리한다.

- Git의 커밋 단위는 앞 단계에서 `README.md`에 정리한 기능 목록 단위로 추가한다.
 - [AngularJS Git Commit Message Conventions](#)을 참고해 커밋 메시지를 작성한다.
- AI 도구를 활용하였다면, `README.md`에 활용한 방식과 코드를 어떻게 수정하였는지, 무엇을 학습하였는지 기록한다.
- 자세한 과제 진행 방법은 프리코스 진행 가이드 문서를 참고한다.

기능 요구 사항

주어진 도시 이름을 입력받아 외부 REST API를 호출해 해당 도시의 날씨 정보를 조회하고, 이를 정리해 반환하는 간단한 웹 서비스를 스프링 프레임워크로 구현한다.

- 사용자는 도시 이름을 입력하면 해당 도시의 위도와 경도를 기반으로 날씨 정보를 조회할 수 있다.
- 최소 다섯 개 이상의 도시는 반드시 지원해야 한다.
 - e.g. Seoul, Tokyo, NewYork, Paris, London
- 도시 이름과 좌표를 매핑하는 자료 구조는 직접 설계한다.
- 서비스는 [Open-Meteo](#) API를 호출하여 아래 정보를 조회한다.
 - 현재 온도
 - 체감 온도
 - 하늘 상태(맑음, 흐림 등)
 - 습도
- 조회한 데이터를 기반으로 간단한 한 줄 요약 문장을 생성하는 기능을 추가한다.
 - e.g. "현재 서울의 기온은 3.4°C이며, 풍속은 5.7m/s입니다. 날씨는 흐림입니다."

프로그래밍 요구 사항

- 자바 코드 컨벤션을 지키면서 프로그래밍한다.
 - 기본적으로 [Google Java Style Guide](#)를 원칙으로 한다.
 - 단, 들여쓰기는 '2 spaces'가 아닌 '4 spaces'로 한다.
- 들여쓰기 단계가 3을 넘지 않도록 구현한다. 2까지만 허용한다.
 - 예를 들어, while문 안의 if문은 들여쓰기 2이다.
 - 힌트: 들여쓰기 단계를 줄이는 좋은 방법은 함수를 분리하는 것이다.
- 함수(또는 메서드)의 길이가 15줄을 넘어가지 않도록 구현한다.
 - 함수는 한 가지 일만 하도록 작성한다.
- else 키워드를 쓰지 않는다.
 - switch문도 허용하지 않는다.
 - 힌트: if문에서 값을 반환하는 방식으로 구현하면 else를 사용하지 않아도 된다.
- 3항 연산자를 쓰지 않는다.
- 정리한 기능 목록이 정상적으로 작동하는지 JUnit 5와 AssertJ로 테스트한다.
 - 테스트 도구 사용법이 익숙하지 않다면 아래 문서를 참고한다.
 - [JUnit 5 User Guide](#)
 - [AssertJ User Guide](#)
 - [AssertJ Exception Assertions](#)
 - [Guide to JUnit 5 Parameterized Tests](#)